

Harness Engineering Intro

Haotian Chen Yulin He Mar. 29 2026

引用

<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

<https://openai.com/index/harness-engineering/>

<https://www.anthropic.com/engineering/harness-design-long-running-apps>

什么是 Harness Engineering?

Harness engineering 不是单独优化模型本身，而是围绕 **agent** 构建一整套执行系统。

它关注的不是“模型这一轮怎么回答”，而是“模型如何在真实任务中持续执行、验证、修复并最终交付结果”。

因此，**harness engineering** 的对象不只是 **prompt**，还包括 **tools**、**middleware**、**context**、**verification**、**feedback loop**、**constraints**、**observability** 等一整套机制。

它和 Prompt Engineering、Context Engineering 有什么区别？

这三者本质上都与模型的上下文管理有关，但关注层级不同。

- **Prompt engineering**：重点是如何**写好**一次输入，让模型在单轮或短链路任务中更好地产生结果。
- **Context engineering**：重点是如何**组织、筛选和注入**模型需要看到的信息，让模型拿到更合适的上下文。
- **Harness engineering**：重点是如何把 **prompt**、**context**、**tools**、验证、修复、约束和反馈机制组织成一个**可持续运行**的系统。

换句话说：

- prompt engineering 解决的是“怎么说”
- context engineering 解决的是“给什么信息”
- harness engineering 解决的是“如何让整个系统跑起来并稳定交付结果”

三者不是互斥关系，而是逐层扩展的关系。

Harness Engineering 是怎么设计的？

上下文来源

将 `repo` 作为一个知识库。在 `AGENTS.md` 提供地图索引，与其他文件配合做到渐进式披露；或者使用中间件扫描 `repo`，直接将文件路径、可用工具、测试命令、边界条件、时间预算等信息注入上下文。

还可以将可观测性内容，如 `logs`、`metrics` 和 `traces` 也纳入上下文，以提升 `agent` 执行的效果。

对于冗长上下文，则可以通过 `context reset` + `structured handoff` 清空上下文，并将上一阶段的 `state`（任务完成情况与后续 `steps`）传递给新启动的 `agent`。

执行结构

基于完整的实现、验证、审核和 `PR` 循环工作流，设计不同阶段的执行结构。

这种结构既可以是 `multi-agent` 协作，例如 `planner` - `generator` - `evaluator`，也可以是 `sole-agent` 内部的分阶段执行步骤，例如 `planning` - `Discovery` - `Build` - `Verify` - `Fix`。

验证结构

在 `agent` 执行后强制进行验证，并进行修复、再验证的循环。（这种设计被统称为 `Ralph Wiggum Loop`）。

验证可以来自其他 `agent` 的多轮审核，也可以是来自通过中间件的 `hook` 触发的自验证。

修复是根据验证结果以及执行轨迹等可观测性信息（例如 `Trace Analyzer Skill`），识别失败模式，调整执行路径；必要时重新设计计划，并重新验证，直到结果通过。

采用一个显式的 `grading criteria` 进行验证通常是良好验证设计的方式。

约束注入

将依赖方向、结构边界、`logging`、`naming`、`file size`、`platform reliability` 等规则写进 `custom linters` 和结构化测试中，由 `agent` 在执行过程中调用这些工具。

在 `planning` 阶段，明确 `task spec` 中的路径、接口设计、`edge cases`、时间预算等信息，促进任务执行的收束。

在 `generator` 和 `evaluator` 之间达成 `sprint contract`，明确这一轮执行怎么样算 `done`、怎么验证 `done`，用什么 `grading criteria` 来约束结果质量。

演进模式

在任务执行前，将经验、`golden principles`、清理规则等内容编码进 `repo` 和工具链。

在任务完成后，根据 `benchmark`、`traces` 和失败模式，持续调整 `system prompt`、`tools`、中间件以及护栏。

随着模型能力增强，`harness` 的设计也可以不断简化。

OpenAI、LangChain 和 Anthropic 的设计有什么区别？

概要

维度	一句话差异
上下文 Context	OpenAI 重构知识与环境，LangChain 注入环境与约束，Anthropic 管长任务交接
执行 Execution	OpenAI 偏工程 workflow，LangChain 偏单 agent 分阶段，Anthropic 偏多 agent 分角色
验证 Verification	OpenAI 偏 mechanical verification，LangChain 偏 self-verification，Anthropic 偏 independent evaluation
修复 Correction	OpenAI 靠 review loop，LangChain 靠 traces 和 middleware，Anthropic 靠 evaluator feedback
约束 Constraints	OpenAI 偏架构规则，LangChain 偏任务约束，Anthropic 偏 contract 和 criteria
演进 Evolution	OpenAI 持续编码经验，LangChain 持续调 harness，Anthropic 持续删 scaffold

具体比较

维度	OpenAI	LangChain	Anthropic
上下文Context	Repository-centered ：把 repo 当成 system of record，用 <code>AGENTS.md</code> 、 <code>ARCHITECTURE.md</code> 、 <code>docs/</code> 做 progressive disclosure；同时把 UI、logs、metrics、traces 做到对 agent 可读	Injection-based ：通过 <code>LocalContextMiddleware</code> 扫描目录、父子路径和可用工具，再把文件路径、测试命令、边界条件、时间预算等上下文直接注入 agent	Reset-based ：长任务中不一直保留冗长上下文，而是用 context reset + structured handoff，把任务状态和 next steps 交给下一阶段

维度	OpenAI	LangChain	Anthropic
执行 Execution	Workflow-driven : 不是单纯多 agent, 而是把 agent 放进实现、验证、review、PR loop 这套工程流程里	Stage-driven : 在单 agent 内部明确 planning / discovery / build / verify / fix 的执行顺序	Role-driven : 显式分成 planner / generator / evaluator, 不同角色负责需求扩展、实现和评估
验证 Verification	Mechanical : tests、lint、CI、type checks、browser automation、observability queries 都被纳入验证链路	Self-checking : 用 <code>PreCompletionChecklistMiddleware</code> 在 agent 退出前强制跑验证, 检查是否真的满足 task spec	Independent : 不让 generator 自评, 而是让 evaluator 独立打分; 同时把标准拆成 explicit grading criteria
修复 Correction	Review-loop : 主要靠 reviewer / PR loop 推动修复, 失败就继续改、继续 review, 直到通过	Trace-driven : 通过 LangSmith traces、Trace Analyzer Skill、LoopDetectionMiddleware 找失败模式, 再调整执行路径	Evaluator-driven : generator 根据 evaluator 的反馈继续 refine, 必要时直接 pivot, 而不是只做局部修补
约束 Constraints	Architecture-first : 把依赖方向、结构边界、logging、naming、file size、platform reliability 等规则写进 custom linters 和 structural tests	Task-first : 重点约束任务执行本身, 比如路径必须精确、接口不能乱改、必须覆盖 edge cases、要在时间预算内收尾	Contract-based : 先形成 sprint contract, 明确这一轮什么算 done、怎么验证 done, 再用 grading criteria 约束结果质量
演进 Evolution	Encoded : 把经验、golden principles、清理规则持续编码进 repo 和工具链, 用 entropy control 定期纠偏	Tuned : 根据 benchmark、traces、failure patterns 持续调 system prompt、tools、middleware; 而且原文明确说有些 guardrails 以后可能不再需要	Simplified : 随着模型变强, 主动删掉不再 load-bearing 的 scaffold, 文章里直接提出 harness simplification